

UNIVERSITÀ DEGLI STUDI DI BARI
ALDO MORO

Drug-Drug Interaction

Progetto di Ingegneria della Conoscenza

Studente: Vincenzo My
Matricola: 774189
Email: v.my1@studenti.uniba.it

Anno Accademico:
2025–2026

Docente:
Prof. Nicola Fanizzi

Repository GitHub:
<https://github.com/vincenzomy03/Drug-Interactions-Predictor.git>

Indice

1	Introduzione	3
1.1	Il problema delle interazioni farmacologiche	3
1.2	Obiettivi del progetto	3
2	Strumenti Utilizzati	5
2.1	Sommario	5
2.2	Ecosistema Python	5
2.2.1	Interfaccia Utente e Dashboard	5
2.2.2	Gestione dei Dati e Machine Learning	5
2.3	Elaborazione Semantica con Owlready2	6
2.4	Motore Logico Prolog Custom	6
2.5	Librerie e Framework	6
3	Algoritmi di Ricerca	7
3.1	Fondamenti Teorici	7
3.2	Allineamento Semantico ed Estrazione delle Feature (DL Reasoning)	7
3.3	Modelli di Machine Learning e Classificazione Sperimentale	8
3.3.1	Logistic Regression (Baseline)	8
3.3.2	Random Forest Classifier (Advanced)	8
3.4	Motore di Inferenza Logica (KBS)	8
4	Implementazione del Motore Logico	10
4.1	Scelta del Ragionamento Logico	10
4.2	Architettura del Motore Prolog Custom	10
4.3	Analisi della Knowledge Base	11
4.4	Rappresentazione dei Termini e Unificazione	11
5	Integrazione dell'Architettura Ibrida	12
5.1	Architettura dell'Integrazione In-Memory	12
5.2	Interazione tra Interfaccia Streamlit e Ragionatore	12
5.3	Conversione dei Dati e Grounding della Knowledge Base	13
6	Utilizzo del Sistema	14
6.1	Struttura del Progetto	14
6.2	Prerequisiti e Installazione	15
6.2.1	Verifica dei Prerequisiti	15
6.2.2	Clonazione e Configurazione	15
6.3	Modalità di Esecuzione	15
6.3.1	Esecuzione della Pipeline CLI	15

6.3.2	Avvio dell'Interfaccia Grafica	16
6.4	Prodotti e Formati di Output	16
6.5	Risoluzione dei Problemi	16
6.5.1	Problemi Comuni	16
7	Valutazione Sperimentale	17
7.1	Protocollo di Valutazione e Dataset	17
7.2	Tuning degli Iperparametri	17
7.3	Risultati del Benchmark Comparativo	18
7.4	Discussione delle Prestazioni ed Efficacia Semantica	18
8	Conclusioni e Sviluppi Futuri	19
8.1	Osservazioni sui Risultati Sperimentali	19
8.2	Conclusioni del Progetto	19
8.3	Limiti Clinici e Disclaimer	20
8.4	Sviluppi Futuri	20

Capitolo 1

Introduzione

1.1 Il problema delle interazioni farmacologiche

Le interazioni farmaco-farmaco (Drug-Drug Interactions, DDI) si verificano quando l'effetto terapeutico, la tossicità o la farmacocinetica di un principio attivo vengono alterati dalla co-somministrazione di un secondo farmaco. Questo fenomeno rappresenta una delle principali sfide della medicina moderna e della farmacovigilanza per diverse ragioni:

1. La diffusione della politerapia: L'invecchiamento della popolazione e l'aumento delle patologie portano una percentuale significativa di pazienti (in particolare geriatrici) ad assumere simultaneamente cinque o più farmaci diversi.
2. Impatto clinico ed economico: Le DDI avverse sono responsabili di una quota rilevante di accessi al pronto soccorso, ospedalizzazioni prolungate, fallimenti terapeutici e, nei casi più gravi, decessi.
3. Esplosione combinatoria: Esistono migliaia di farmaci approvati sul mercato. Testare clinicamente ogni possibile coppia di farmaci per identificare interazioni avverse è impraticabile a causa di vincoli etici, economici e di tempo. I trial clinici preliminari spesso non riescono a rilevare interazioni rare o a lungo termine, che emergono solo nella fase di post-marketing. La predizione computazionale delle DDI e, soprattutto, la comprensione del meccanismo causale alla base di esse sono quindi cruciali per supportare le decisioni cliniche e prevenire eventi avversi.

La predizione computazionale e la comprensione del meccanismo di tali interazioni costituiscono un caso di studio ideale per l'Ingegneria della Conoscenza per due ragioni principali: in primo luogo, il superamento dei limiti del Machine Learning puro tramite un approccio di AI Ibrida, dove il ragionamento simbolico di un motore Prolog applica regole deduttive ai fatti estratti dall'ontologia per fornire la spiegabilità biologica e causale preclusa ai modelli black-box; in secondo luogo, l'arricchimento semantico, che permette di addestrare i modelli predittivi direttamente sulle proprietà logiche e biochimiche dedotte dall'ontologia anziché su semplici nomi di farmaci, migliorando l'accuratezza e la capacità di generalizzare su molecole mai viste prima.

1.2 Obiettivi del progetto

L'obiettivo principale di questo lavoro consiste nella progettazione e nello sviluppo di un sistema intelligente ibrido per la predizione e la spiegazione delle interazioni farmaco-

farmaco. Il sistema si propone di superare i limiti dei modelli statistici tradizionali integrando sinergicamente le tecniche di Machine Learning con gli strumenti e i formalismi propri del Web Semantico e dei Sistemi Basati sulla Conoscenza (KBS). L'architettura software mira a realizzare una pipeline completa che parte dall'acquisizione del dato clinico grezzo fino ad arrivare alla visualizzazione di una diagnosi predittiva trasparente e giustificabile dal punto di vista medico.

Sul piano dell'ingegnerizzazione della conoscenza, il primo obiettivo operativo riguarda l'allineamento semantico tra i dati clinici reali e la base di conoscenza formale rappresentata dall'ontologia biologica ChEBI. Il sistema punta a risolvere i problemi di sinonimia e variazione terminologica dei farmaci, estraendone in modo automatico i ruoli funzionali e le classi chimiche di appartenenza tramite il ragionamento automatico sulle gerarchie della Knowledge Base.

Un altro obiettivo centrale è la dimostrazione empirica del valore aggiunto fornito dall'arricchimento semantico nel Machine Learning. Il progetto mira a costruire una rappresentazione tabulare in cui le feature dei farmaci non siano semplici identificativi testuali o proprietà chimiche a basso livello, bensì i ruoli logici dedotti dall'ontologia. Questo consente di addestrare classificatori capaci di generalizzare i pattern di interazione a un livello di astrazione più elevato, confrontando le prestazioni di un modello avanzato basato su Random Forest con una baseline lineare per convalidare l'efficacia predittiva di tale rappresentazione.

Infine, il progetto persegue l'obiettivo dell'esplicabilità (Explainable AI) attraverso l'implementazione di un interprete Prolog scritto interamente in Python. Questo motore inferenziale ha lo scopo di eseguire un ragionamento deduttivo a runtime sulle coppie di farmaci analizzate, identificando sovrapposizioni logiche nei loro meccanismi d'azione ed evidenziando ruoli precedentemente classificati come critici. L'intero flusso logico e statistico viene infine reso accessibile all'utente finale attraverso l'interfaccia grafica, che unisce la predizione probabilistica del livello di rischio con una traccia trasparente del ragionamento logico eseguito dal sistema.

Capitolo 2

Strumenti Utilizzati

2.1 Sommario

Il sistema di predizione delle interazioni farmacologiche è stato sviluppato seguendo un approccio ibrido che integra le tecnologie del Web Semantico, il Machine Learning statistico e la programmazione logica. La scelta di utilizzare Python come unico ecosistema di sviluppo risponde alla necessità di garantire la massima portabilità del software e di evitare dipendenze esterne complesse, come motori Prolog esterni. A tal fine, le capacità di modellazione e classificazione statistica di Scikit-Learn e la gestione ontologica tramite Owlready2 sono affiancate da un motore di inferenza logica dichiarativo (Prolog) implementato direttamente all'interno del codice sorgente. Questo consente di ottenere sia le performance predittive del Machine Learning sia la trasparenza esplicativa del paradigma simbolico in un'unica applicazione integrata.

2.2 Ecosistema Python

Python costituisce l'infrastruttura principale per l'orchestrazione dell'intera pipeline dei dati. La versatilità del linguaggio ha permesso di gestire tutte le fasi del ciclo di vita dell'applicazione, a partire dall'estrazione semantica delle caratteristiche dei farmaci fino all'addestramento dei modelli e alla presentazione interattiva delle deduzioni logiche.

2.2.1 Interfaccia Utente e Dashboard

Per la realizzazione dell'interfaccia utente è stato impiegato Streamlit, un framework orientato allo sviluppo rapido di applicazioni web per la scienza dei dati. A differenza delle librerie GUI desktop tradizionali, Streamlit consente di generare un'interfaccia web interattiva, reattiva e ottimizzata per la visualizzazione di grafici, metriche e codice. Streamlit viene utilizzato per ricevere l'input dell'utente, mostrare in tempo reale la predizione probabilistica del modello di Machine Learning e visualizzare l'esecuzione passo-passo delle query logiche eseguite dal ragionatore Prolog.

2.2.2 Gestione dei Dati e Machine Learning

La manipolazione dei dataset e l'addestramento dei classificatori si basano su due librerie fondamentali di Python. La libreria Pandas viene impiegata per pulire le interazioni

cliniche grezze provenienti dal database DDInter e per eseguire le operazioni di join relazionale con le caratteristiche semantiche dei farmaci. Successivamente, Scikit-Learn viene utilizzato per vettorizzare i ruoli estratti tramite tecniche di Bag-of-Words, codificare le classi di rischio e addestrare i modelli di classificazione Random Forest e Logistic Regression, gestendo anche la cross-validation e la ricerca degli iperparametri ottimali (GridSearchCV).

2.3 Elaborazione Semantica con Owlready2

L'interazione con l'ontologia biologica ChEBI avviene tramite Owlready2, un modulo Python progettato per l'orientamento agli oggetti su ontologie OWL. Owlready2 consente di caricare localmente l'ontologia in formato OWL, accedere alle classi chimiche e ai sinonimi e navigare la gerarchia delle classi. La libreria permette di calcolare la chiusura transitiva degli antenati di un farmaco per estrarre tutti i ruoli biologici attivi ereditati lungo la tassonomia, traducendo la conoscenza formale dell'ontologia in caratteristiche numeriche digeribili dai modelli statistici.

2.4 Motore Logico Prolog Custom

Invece di affidarsi a motori di inferenza esterni (come SWI-Prolog) tramite librerie di collegamento quali pyswip (che avrebbero richiesto l'installazione separata del compilatore Prolog sul sistema ospite), il progetto implementa un interprete Prolog custom scritto in Python. Questo interprete implementa le strutture dati fondamentali per la programmazione logica, comprese le variabili logiche, le regole di unificazione ricorsiva, la ridenominazione dei termini per evitare collisioni di scope e il meccanismo di backtracking per la risoluzione dei goal. In questo modo, il sistema è in grado di eseguire query logiche dichiarative direttamente in memoria a runtime all'interno dell'applicazione Streamlit.

2.5 Librerie e Framework

Libreria	Utilizzo principale nel progetto
streamlit	Sviluppo della dashboard web e presentazione interattiva dei risultati
pandas	Pulizia, manipolazione del dataset e fusione dei dati clinici con la KB
scikit-learn	Vettorizzazione delle feature, tuning e addestramento dei modelli di Machine Learning
owlready2	Caricamento dell'ontologia ChEBI, allineamento terminologico e DL reasoning
joblib	Serializzazione su disco dei modelli addestrati e del vettorizzatore
os / time	Gestione dei percorsi dei file e misurazione delle performance temporali

Tabella 2.1: Principali dipendenze del progetto

Capitolo 3

Algoritmi di Ricerca

3.1 Fondamenti Teorici

La risoluzione del problema delle interazioni farmacologiche (DDI) viene qui affrontata attraverso un paradigma di Intelligenza Artificiale Ibrida che combina l'apprendimento statistico (subsimbolico) con il ragionamento logico-deduttivo (simbolico). Dal punto di vista teorico, il problema della predizione delle DDI può essere formalizzato come un compito di classificazione supervisionata su coppie di entità.

Dato un insieme di coppie di farmaci $P = \{(d_a, d_b)\}$ e un insieme ordinato di classi target $Y = \{\text{Minor}, \text{Moderate}, \text{Major}\}$, l'obiettivo è apprendere una funzione di mappatura $f : P \rightarrow Y$ in grado di minimizzare l'errore di classificazione. A differenza dei modelli tradizionali che trattano i farmaci come identificativi atomici privi di struttura, questo sistema associa a ciascun farmaco d una porzione di conoscenza formale estratta da un'ontologia espressa in Logica Descrittiva (DL). La conoscenza di background associata a un farmaco viene descritta come l'insieme dei suoi ruoli biochimici e delle sue classi chimiche genitrici, consentendo al modello di ragionare sulle proprietà funzionali condivise piuttosto che sulla mera associazione statistica dei nomi dei farmaci.

3.2 Allineamento Semantico ed Estrazione delle Feature (DL Reasoning)

L'arricchimento semantico del dataset avviene mappando i farmaci testuali sul grafo della Knowledge Base ChEBI. Lo script `extract_features.py` implementa una strategia di allineamento a due livelli: un match esatto basato su dizionari in memoria indicizzanti label e sinonimi dell'ontologia (con complessità temporale $O(1)$) e, in caso di fallimento, una ricerca parziale fuzzy per identificare sottostringhe comuni.

Una volta individuato il nodo ontologico corrispondente al farmaco, viene eseguito il ragionamento formale per estrarre la chiusura transitiva delle sue proprietà.

Definizione 3.1 (Estrazione dei Ruoli Semantici). Sia C_d il concetto ontologico associato al farmaco d . L'insieme dei ruoli biologici attivi R_d ereditati dal farmaco è definito dall'unione delle restrizioni esistenziali ereditate lungo la gerarchia di sussunzione dell'ontologia:

$$R_d = r \in \mathcal{C}_{roles} \mid \exists A \text{ t.c. } C_d \sqsubseteq A \text{ e } A \sqsubseteq \exists \text{has_role.}r$$

dove $\mathcal{C}_{roles}$ rappresenta la classe dei ruoli in ChEBI e la relazione di ruolo è identificata dalla proprietà oggettiva `has_role` (codificata nell'ontologia come 'RO_0000087').

Questo calcolo permette di catturare non solo le proprietà dichiarate direttamente per il farmaco specifico, ma anche tutta la conoscenza implicita ereditata filogeneticamente dalla sua classe chimica o funzionale.

3.3 Modelli di Machine Learning e Classificazione Sperimentale

Una volta estratte le feature semantiche per ciascun farmaco della coppia, la matrice dei dati viene assemblata in `build_dataset.py`. I ruoli testuali uniti della coppia di farmaci vengono poi convertiti in un vettore binario sparso attraverso una vettorizzazione di tipo Bag-of-Words, in cui ogni coordinata indica la presenza o l'assenza di un determinato ruolo ChEBI nell'interazione. Il benchmark sperimentale descritto in `train_models.py` valuta due modelli principali per risolvere il task di classificazione multiclasse.

3.3.1 Logistic Regression (Baseline)

La regressione logistica viene impiegata come baseline lineare di riferimento. Questo modello calcola la probabilità di appartenenza a ciascuna classe di rischio tramite una combinazione lineare delle feature vettoriali normalizzata con la funzione softmax. È particolarmente utile per valutare la separabilità lineare dello spazio delle feature semantiche, sebbene non sia in grado di catturare le interazioni non lineari complesse tra i ruoli dei due farmaci.

3.3.2 Random Forest Classifier (Advanced)

La Random Forest viene selezionata come modello principale per via della sua natura non lineare e robusta. Essendo un classificatore ensemble basato sul bagging di alberi di decisione indipendenti, essa si adatta perfettamente a vettori di feature sparsi e binari. Ogni albero viene addestrato su un sottoinsieme casuale dei dati e delle feature, riducendo drasticamente il rischio di overfitting e permettendo di catturare le relazioni condizionali del tipo "se il farmaco A ha il ruolo X e il farmaco B ha il ruolo Y, allora il rischio è elevato". Gli iperparametri del modello (come il numero di stimatori e la profondità massima) vengono ottimizzati tramite `GridSearchCV` applicando una partizione in 5-Fold Cross-Validation per garantire la generalizzabilità dei risultati.

3.4 Motore di Inferenza Logica (KBS)

Per fornire spiegazioni logico-deduttive comprensibili, il sistema non si affida esclusivamente all'output probabilistico del Machine Learning, ma interroga un sistema basato sulla conoscenza (KBS) implementato da zero in `prolog_interpreter.py`. L'interprete implementa un motore a programmazione logica dichiarativa basato sulla risoluzione SLD (Selected Literal Definite clause resolution).

Definizione 3.2 (Unificazione Logica). Dati due termini t_1 e t_2 , l'algoritmo di unificazione cerca la sostituzione minimale θ (ovvero un insieme di associazioni tra variabili e costanti)

tale che $t_1\theta = t_2\theta$. Se tale sostituzione esiste, i due termini sono unificabili, altrimenti l'unificazione fallisce restituendo un valore nullo.

La risoluzione dei goal logici avviene tramite una ricerca in profondità (Depth-First Search) con backtracking automatico, gestita ricorsivamente nel metodo `'_solve'`. Per prevenire la collisione di scope tra le variabili durante la risoluzione ricorsiva delle regole, ogni clausola viene rinominata a runtime assegnando un suffisso numerico incrementale alle sue variabili.

Il KBS viene interrogato nel modulo `app.py` istanziando due regole deduttive fondamentali:

```
1 interact(X, Y, Role) :-  
2     has_role(X, Role),  
3     has_role(Y, Role).  
4  
5 high_risk_interaction(X, Y, Role) :-  
6     interact(X, Y, Role),  
7     critical_role(Role).
```

Listing 3.1: Regole Prolog per il calcolo del rischio di interazione

Il motore riceve come fatti (grounding) i ruoli estratti a runtime dall'ontologia per i due farmaci selezionati e verifica se esiste una sovrapposizione logico-funzionale. Qualora la query `interact` o `high_risk_interaction` abbia successo, il sistema deduce e mostra all'utente l'esatta catena causale dell'interazione identificando il ruolo biologico condiviso.

Capitolo 4

Implementazione del Motore Logico

4.1 Scelta del Ragionamento Logico

L'implementazione del ragionamento deduttivo all'interno del sistema non si affida a compilatori Prolog esterni (come SWI-Prolog), bensì a un interprete logico proprietario sviluppato interamente in Python. Questa scelta architetturale è motivata dalla volontà di garantire la massima portabilità dell'applicazione, eliminando dipendenze di sistema a basso livello e semplificando la procedura di installazione. Inoltre, l'integrazione di un motore logico scritto nello stesso linguaggio dell'applicazione principale consente una comunicazione diretta e a bassissimo overhead tra le strutture dati del Machine Learning (come i DataFrame di Pandas) e il motore inferenziale.

Il paradigma dichiarativo implementato consente di separare nettamente la conoscenza (i fatti biologici dei farmaci e le regole di interazione) dall'algoritmo di controllo che esegue le deduzioni. In questo modo è stato possibile codificare in modo naturale e leggibile le regole del dominio clinico, fornendo una spiegazione esplicativa trasparente (Explainable AI) altrimenti impossibile da ottenere tramite l'uso esclusivo di modelli statistici black-box.

4.2 Architettura del Motore Prolog Custom

Il cuore del sistema basato sulla conoscenza (KBS) risiede in un modulo che emula il comportamento di un interprete Prolog standard attraverso la programmazione orientata agli oggetti. L'architettura del motore è composta da elementi fondamentali che gestiscono la rappresentazione dei termini, l'unificazione e la ricerca nello spazio delle prove.

La rappresentazione delle variabili logiche è affidata alla classe `Var`, la quale identifica le entità simboliche da istanziare durante la risoluzione (es. `?X` o `?Role`). La classe implementa i metodi per verificare l'uguaglianza logica ed evitare collisioni di nomi.

La risoluzione dei goal logici viene orchestrata dalla classe `Prolog`, che memorizza la base di conoscenza sotto forma di una lista di clausole composte da un'intestazione (head) e un corpo (body). La ricerca delle soluzioni avviene tramite un algoritmo ricorsivo di backtracking che effettua una ricerca in profondità (Depth-First Search) sull'albero di prova SLD. Per garantire la correttezza della risoluzione logica ed evitare che variabili con lo stesso nome in clausole diverse interferiscano tra loro, il motore implementa un meccanismo di ridenominazione dinamica che assegna un suffisso numerico univoco alle variabili di ciascuna regola durante l'espansione.

4.3 Analisi della Knowledge Base

La Knowledge Base gestisce la memorizzazione e l'inferenza di due categorie distinte di conoscenza: una base fattuale (fatti a priori e fatti a runtime) e un insieme di regole di deduzione logica.

I fatti statici definiscono i ruoli biologici ChEBI ritenuti clinicamente critici, associandoli al predicato `critical_role/1`. Questi ruoli identificano classi farmacologiche particolarmente sensibili (es. agenti anticoagulanti, immunosoppressori o depressori del sistema nervoso centrale). I fatti dinamici vengono invece asseriti a runtime non appena l'utente seleziona la coppia di farmaci da analizzare. Il sistema estrae i ruoli corrispondenti dal file di feature e asserisce i predicati `has_role/2` (es. `has_role(abacavir, ruolo_specifico)`).

Le regole inferenziali sono codificate in forma dichiarativa all'interno del sistema e rappresentano il nucleo logico per determinare la sovrapposizione biologica e la severità delle interazioni:

Regola di Interazione Generale (`interact`): Definisce che due farmaci X e Y interagiscono se condividono il medesimo ruolo biologico o chimico.

```
1 interact(X, Y, Role) :- has_role(X, Role), has_role(Y, Role).
```

Regola di Rischio Elevato (`high_risk_interaction`): Stabilisce che l'interazione tra X e Y è ad alto rischio se i due farmaci condividono un ruolo che è classificato a priori come critico.

```
1 high_risk_interaction(X, Y, Role) :- interact(X, Y, Role),  
    critical_role(Role).
```

4.4 Rappresentazione dei Termini e Unificazione

I termini logici all'interno dell'interprete sono mappati direttamente su tipi nativi di Python: le costanti atomiche sono rappresentate da stringhe, mentre le strutture composte (i predicati) sono modellate come tuple. Questa corrispondenza semplifica il pattern matching.

Il processo di unificazione è implementato come una funzione ricorsiva che confronta due termini e restituisce un dizionario contenente le sostituzioni correnti. L'algoritmo analizza ricorsivamente le tuple e associa le variabili logiche a termini specifici. La funzione di `walk` risolve le catene di sostituzione per garantire che il valore corrente di una variabile sia sempre recuperato prima di tentare una nuova unificazione. Se in qualsiasi momento dell'ispezione ricorsiva si rileva una discrepanza strutturale o di valore tra i termini, l'unificazione fallisce immediatamente, forzando l'algoritmo di risoluzione a retrocedere (`backtrack`) per esplorare percorsi alternativi.

Capitolo 5

Integrazione dell'Architettura Ibrida

5.1 Architettura dell'Integrazione In-Memory

A differenza dei tradizionali sistemi accademici che si interfacciano a compilatori logici esterni mediante librerie di binding e Foreign Function Interface (FFI), questo sistema adotta un approccio di integrazione completamente in-memory. Poiché il motore Prolog è scritto nativamente in Python, non è richiesta alcuna configurazione di variabili d'ambiente di sistema o l'installazione di runtime aggiuntivi. La classe Prolog viene istanziata direttamente come un normale oggetto Python, garantendo l'assoluta portabilità cross-platform e l'immunità da problemi legati al garbage collection concorrente dei due linguaggi.

Questa scelta progettuale garantisce una type safety nativa. Le strutture dati passate al motore logico (come tuple e stringhe) e i risultati restituiti (sotto forma di dizionari di sostituzioni) non subiscono alcun processo di serializzazione o marshalling binario, eliminando l'overhead computazionale tipico della comunicazione inter-processo o dei bridge FFI.

5.2 Interazione tra Interfaccia Streamlit e Ragionatore

Il modulo dell'interfaccia utente in `app.py` agisce come orchestratore del flusso di esecuzione, coordinando il caricamento iniziale dei modelli e la chiamata a runtime al motore deduttivo. Il ciclo di vita dell'interazione segue una sequenza deterministica attivata dall'utente:

Dopo che l'utente ha selezionato i due farmaci e avviato l'analisi, il sistema interroga il DataFrame contenente le feature ontologiche pre-estratte per recuperare le liste di ruoli associate. Viene quindi inizializzato un nuovo oggetto Prolog vuoto e caricato con le regole logiche di base e con i fatti statici relativi ai ruoli ad alto rischio (`critical_role/1`). La Knowledge Base viene infine popolata a runtime con i ruoli specifici della coppia di farmaci analizzati.

Una volta completato il grounding, l'applicazione interroga il motore logico tramite il metodo `query` per ottenere le deduzioni. Streamlit si occupa infine di visualizzare i risultati, mostrando la traccia del codice Prolog simulato, i fatti asseriti in memoria e l'output della deduzione sotto forma di messaggi informativi e box di allerta contestuali.

5.3 Conversione dei Dati e Grounding della Knowledge Base

Nonostante l'integrazione avvenga nello stesso spazio di memoria, è necessario effettuare una normalizzazione sintattica per convertire le rappresentazioni testuali dei farmaci e dei ruoli in atomi logici conformi alla sintassi Prolog.

I nomi dei farmaci selezionati nell'interfaccia grafica vengono convertiti in minuscolo e i relativi spazi bianchi sostituiti da caratteri di sottolineatura (es. "Abacavir" diventa l'atomo "abacavir"). Questo processo previene errori di parsing nell'unificazione. Successivamente, le stringhe contenenti i ruoli ChEBI dei farmaci (originariamente memorizzate nel database e separate dal carattere pipe) vengono divise e ripulite. Per ogni ruolo identificato, viene assertito a runtime un fatto strutturato come tupla Python:

```
("has_role", atom_farmaco, atom_ruolo)
```

il quale viene inserito nella lista delle clausole del motore logico.

Al termine del ragionamento deduttivo, le risposte fornite dal generatore di query di Prolog sono rappresentate da dizionari che associano gli oggetti Var ai corrispondenti termini unificati. Il sistema applica la funzione di walk per risolvere ricorsivamente le variabili logiche e ricondurle a stringhe Python native, che vengono poi utilizzate direttamente per popolare dinamicamente la dashboard utente.

Capitolo 6

Utilizzo del Sistema

6.1 Struttura del Progetto

Il progetto è organizzato in una struttura modulare che separa in modo netto l'acquisizione e la pulizia dei dati grezzi, l'estrazione semantica tramite l'ontologia, l'addestramento dei modelli di Machine Learning e l'interfaccia utente finale. L'albero delle cartelle e dei file del repository si presenta come segue:

```
Drug-Interactions-Predictor/
|-- requirements.txt      # Dipendenze Python del progetto
|-- README.md            # Manuale operativo e documentazione rapida
|
|-- data/                # Dati del progetto
|   |-- ontologies/      # Contiene il file dell'ontologia chebi.owl
|   |-- raw/              # Dataset clinici originali non elaborati
|   |-- processed/       # Dataset puliti e feature semantiche estratte
|
|-- models/              # File dei modelli serializzati (.pkl)
|
|-- docs/                # Documentazione del progetto
|
|-- src/                 # Codice sorgente Python
|   |-- extract_features.py # Estrazione feature semantiche da ChEBI
|   |-- build_dataset.py   # Join tra dati clinici ed estrazioni semantiche
|   |-- train_models.py    # Addestramento e benchmark di cross-validation
|   |-- save_final_model.py # Addestramento e serializzazione del modello finale
|   |-- evaluate_atc_subsets.py # Valutazione comparativa sui sotto-dataset ATC
|   |-- ontology_mapper.py # Utility di test caricamento ontologia e query
|   |-- prolog_interpreter.py # Interprete Prolog custom scritto in Python
|   |-- app.py            # Dashboard grafica interattiva in Streamlit
```

6.2 Prerequisiti e Installazione

6.2.1 Verifica dei Prerequisiti

A differenza di altre soluzioni che richiedono librerie di terze parti per interfacciarsi a motori esterni, questo sistema necessita unicamente di un ambiente di esecuzione Python 3.9 o superiore. Non è richiesta l'installazione di alcun software aggiuntivo o compilatore Prolog proprietario, rendendo il sistema autosufficiente ed estremamente leggero.

6.2.2 Clonazione e Configurazione

La configurazione iniziale prevede la clonazione del codice sorgente, la creazione di un ambiente virtuale per isolare le librerie e l'installazione delle dipendenze necessarie. La procedura da eseguire a riga di comando si articola in passaggi precisi:

Clonazione del repository e posizionamento nella directory principale:

```
1 git clone <URL_DEL_REPO>
2 cd Drug-Interactions-Predictor
```

Creazione e attivazione dell'ambiente virtuale (su sistema operativo Windows):

```
1 python -m venv venv
2 .\venv\Scripts\Activate.ps1
```

Installazione dei pacchetti Python specificati nel file requirements.txt:

```
1 pip install -r requirements.txt
```

In aggiunta a queste operazioni, poiché il file dell'ontologia ChEBI supera le dimensioni consentite da Git, è necessario scaricare manualmente o tramite terminale il file chebi.owl dall'European Bioinformatics Institute e posizionarlo sotto la cartella data/ontologies/.

6.3 Modalità di Esecuzione

Il ciclo di esecuzione del sistema è suddiviso in due fasi: una fase preliminare basata su riga di comando per l'elaborazione dei dati e l'addestramento dei modelli, e una fase interattiva basata su interfaccia web.

6.3.1 Esecuzione della Pipeline CLI

La rigenerazione completa del dataset arricchito semanticamente e dei modelli predittivi viene effettuata eseguendo in sequenza i moduli presenti nella cartella src:

Avvio dell'estrazione semantica dall'ontologia:

```
1 python src/extract_features.py
```

Fusione dei dati estratti con il dataset clinico grezzo:

```
1 python src/build_dataset.py
```

Valutazione sperimentale dei modelli in cross-validation:

```
1 python src/train_models.py
```

Addestramento finale e serializzazione su disco del modello ottimizzato:

```
1 python src/save_final_model.py
```


6.3.2 Avvio dell'Interfaccia Grafica

Una volta prodotti i file dei modelli nella cartella `models`, è possibile avviare l'applicazione web interattiva per visualizzare le analisi e le deduzioni logiche in tempo reale. Il server Streamlit viene avviato tramite il comando:

```
1 streamlit run src/app.py
```

L'applicazione apre automaticamente una scheda all'interno del browser all'indirizzo <http://localhost:8501>.

6.4 Prodotti e Formati di Output

Durante l'esecuzione della pipeline dei dati, il sistema genera file intermedi e artefatti di classificazione. Sotto la cartella dei dati elaborati (`data/processed/`) vengono prodotti file CSV contenenti i farmaci mappati, i ruoli ereditati ed il dataset finale di addestramento. All'interno della cartella `models/` sono salvati in formato serializzato (`.pkl`) i pesi del classificatore Random Forest, il dizionario di vettorizzazione dei testi semantici ed il codificatore delle classi di rischio. L'output conclusivo per l'utente finale è rappresentato dall'interfaccia interattiva che mostra la severità stimata dell'interazione e il tracciato del codice Prolog (Prolog Trace) con le deduzioni logiche ricavate dal ragionatore.

6.5 Risoluzione dei Problemi

6.5.1 Problemi Comuni

Mancanza dei file del modello all'avvio della GUI: Qualora l'applicazione Streamlit mostri un errore relativo al caricamento dei file `.pkl`, è necessario verificare di aver precedentemente eseguito lo script di salvataggio del modello finale (`save_final_model.py`) per generare gli artefatti mancanti nella cartella dei modelli.

Tempi di caricamento dell'ontologia o memoria insufficiente: L'ontologia ChEBI è caratterizzata da dimensioni elevate e richiede una quantità significativa di RAM durante la prima fase di allineamento e ragionamento gerarchico. In caso di rallentamenti eccessivi su macchine con risorse hardware limitate, si consiglia di attendere il completamento dello script di estrazione senza interrompere il processo, poiché la pipeline è progettata per salvare i risultati intermedi in file CSV in modo che i successivi avvii della dashboard Streamlit non debbano ricaricare l'ontologia OWL da zero.

Capitolo 7

Valutazione Sperimentale

7.1 Protocollo di Valutazione e Dataset

La valutazione delle capacità predittive del sistema è stata condotta attraverso una pipeline di testing sistematico implementata in `train_models.py`. Il dataset finale pre-elaborato, generato dall'unione delle interazioni cliniche e delle feature semantiche dell'ontologia, è composto da un campione statisticamente significativo di 106.839 coppie di farmaci interagenti. La vettorizzazione dei ruoli estratti ha generato uno spazio delle feature composto da 533 attributi semantici binari.

Le tre classi di rischio target per la classificazione sono:

- **Minor**: Interazione a basso rischio clinico.
- **Moderate**: Interazione a rischio moderato, richiede monitoraggio.
- **Major**: Interazione critica, co-somministrazione sconsigliata.

Per garantire la robustezza delle metriche ed evitare distorsioni dovute all'overfitting o a una favorevole suddivisione dei dati di addestramento, è stato adottato un protocollo di validazione incrociata a 5 fold (5-Fold Cross-Validation). Le metriche principali raccolte sono l'Accuracy media, che esprime la percentuale complessiva di predizioni corrette, e il Macro F1-Score medio, selezionato come metrica di riferimento per via della sua capacità di valutare in modo bilanciato le prestazioni su tutte le classi indipendentemente dal loro sbilanciamento nel dataset.

7.2 Tuning degli Iperparametri

Prima della fase di benchmark comparativo, è stata eseguita una ricerca esaustiva sulla griglia dei parametri (GridSearchCV) per ottimizzare le prestazioni del modello Random Forest. La ricerca ha valutato diverse combinazioni per il numero di alberi decisionali (`n_estimators`) e per la loro profondità massima (`max_depth`).

La ricerca ha identificato come configurazione ottimale un numero di stimatori pari a 100 con una profondità degli alberi non limitata (`max_depth = None`). Questa combinazione ha permesso di raggiungere il miglior punteggio di Macro F1 in fase di validazione, pari a 0,6518. Il modello così ottimizzato è stato successivamente utilizzato nel confronto con la baseline lineare e infine serializzato su disco per l'integrazione con la GUI.

7.3 Risultati del Benchmark Comparativo

I risultati ottenuti dalla valutazione incrociata mostrano differenze sostanziali tra il modello lineare di baseline e il modello ensemble avanzato basato su Random Forest.

Tabella 7.1: Risultati comparativi in 5-Fold Cross-Validation

Modello	Accuracy Media	Macro F1-Score Medio
Baseline (Logistic Regression)	0,5617 ($\pm 0,0529$)	0,4699 ($\pm 0,0371$)
Advanced (Tuned Random Forest)	0,7702 ($\pm 0,0350$)	0,6518 ($\pm 0,0290$)

Come si evince dai dati riportati nella Tabella 7.1, la regressione logistica lineare registra prestazioni modeste, attestandosi su un'accuracy del 56,17% e un Macro F1-Score inferiore al 50%. Il classificatore Random Forest ottimizzato evidenzia un netto incremento prestazionale, raggiungendo un'accuracy del 77,02% (+20,85% rispetto alla baseline) e un Macro F1-Score di 0,6518 (+18,19% rispetto alla baseline), confermando una stabilità elevata e deviazioni standard contenute in tutti i fold.

7.4 Discussione delle Prestazioni ed Efficacia Semantica

L'analisi comparativa fornisce importanti indicazioni sulla natura del dominio delle interazioni farmacologiche e sull'efficacia dell'arricchimento semantico basato sulla Knowledge Base ChEBI.

Il marcato divario prestazionale tra la regressione logistica e la Random Forest risiede nella non-linearità intrinseca delle interazioni farmacologiche. La regressione logistica tenta di tracciare un iperpiano di separazione lineare, assegnando un peso individuale a ciascun ruolo semantico (ad esempio, penalizzando singolarmente la presenza di un ruolo anticoagulante). Tuttavia, l'interazione clinica reale è un fenomeno strettamente combinatorio: l'insorgenza di un effetto avverso non dipende dalla presenza isolata di una proprietà in un singolo farmaco, ma dalla co-occorrenza sinergica di determinati ruoli in entrambi i farmaci (ad esempio, se il farmaco A è un anticoagulante e il farmaco B è un inibitore dell'aggregazione piastrinica). Grazie alla sua struttura ad alberi di decisione, la Random Forest è in grado di apprendere naturalmente queste regole logico-condizionali e le interazioni tra feature senza la necessità di ingegnerizzare manualmente termini di interazione complessi.

I risultati dimostrano infine la validità dell'approccio basato sull'Ingegneria della Conoscenza: l'utilizzo esclusivo di feature semantiche astratte estratte dall'ontologia consente al modello di raggiungere un'accuratezza del 77%, fornendo al contempo un vocabolario di termini logici leggibili sia dai modelli statistici sia dal motore di inferenza deduttivo Prolog per la generazione di spiegazioni cliniche trasparenti.

Capitolo 8

Conclusioni e Sviluppi Futuri

8.1 Osservazioni sui Risultati Sperimentali

L'analisi empirica condotta su oltre 100.000 interazioni farmacologiche ha confermato la validità dell'ipotesi scientifica alla base di questo lavoro: l'iniezione di conoscenza semantica formale migliora sensibilmente la capacità predittiva dei sistemi intelligenti applicati alla bioinformatica. I test effettuati hanno evidenziato come la Random Forest, arricchita con le feature semantiche dedotte dall'ontologia ChEBI, sia in grado di catturare le complesse relazioni combinatorie e non lineari dei ruoli biologici, superando del 20,85% l'accuratezza del modello lineare di baseline.

Parallelamente, la validazione a runtime del ragionatore Prolog ha dimostrato l'efficacia del paradigma simbolico nell'affiancare i modelli probabilistici. L'interprete logico è stato in grado di giustificare le predizioni del modello statistico identificando le specifiche sovrapposizioni biochimiche (ad esempio, evidenziando i ruoli critici condivisi come agenti anticoagulanti o immunosoppressori). Questo approccio ibrido delinea un profilo di intelligenza artificiale spiegabile (XAI), in cui l'accuratezza predittiva del Machine Learning non viene sacrificata a scapito della trasparenza clinica, ma le due componenti cooperano in modo complementare.

8.2 Conclusioni del Progetto

Il progetto ha conseguito con successo tutti gli obiettivi prefissati, fornendo un esempio concreto di come i Sistemi Basati sulla Conoscenza (KBS) e il Semantic Web possano integrarsi con i moderni modelli di apprendimento automatico. L'architettura software realizzata dimostra che è possibile implementare una pipeline complessa di estrazione e allineamento semantico garantendo al contempo ottime prestazioni e una manutenzione agevole del codice.

Dal punto di vista tecnico e didattico, l'introduzione di un motore Prolog custom scritto in Python ha risolto brillantemente i problemi di portabilità legati a compilatori esterni, dimostrando come i meccanismi fondamentali della programmazione logica (quali unificazione e backtracking) possano essere ricreati e integrati in contesti applicativi moderni come dashboard web reattive. La modularità del repository facilita l'estensione del sistema con nuove regole deduttive o differenti modelli predittivi, ponendo solide basi per lo sviluppo di sistemi di supporto alle decisioni cliniche più avanzati.

8.3 Limiti Clinici e Disclaimer

Nonostante l'elevata accuratezza statistica registrata dal classificatore, è fondamentale delineare i confini applicativi del sistema sul piano pratico.

Questo sistema è un prototipo accademico. Le predizioni generate dal modello statistico e le deduzioni logiche ricavate dal ragionatore non tengono conto di variabili cliniche fondamentali del paziente reale, quali il dosaggio terapeutico (farmacocinetica e farmacodinamica), la presenza di co-morbilità concomitanti, l'età, il genere e lo stato di funzionalità epatica o renale. Pertanto, il software non deve in alcun caso essere utilizzato come strumento di supporto decisionale per la prescrizione o la modifica di terapie su pazienti reali al di fuori di un ambiente di simulazione biomedica controllato.

8.4 Sviluppi Futuri

Una prima linea di sviluppo, parzialmente affrontata in questo lavoro attraverso lo script sperimentale `evaluate_atc_subsets.py`, riguarda la valutazione e l'allineamento dei farmaci con la classificazione ATC (Anatomical Therapeutic Chemical). Nel sistema attuale, i dati clinici sono stati suddivisi in base alle categorie ATC principali (es. Sangue, Dermatologici, Sistema Respiratorio) per analizzare la tenuta del modello su classi specifiche di farmaci. Come sviluppo futuro, si prevede l'integrazione formale e gerarchica di un'ontologia ATC o SNOMED-CT direttamente nella base di conoscenza, al fine di arricchire ulteriormente il profilo semantico dei farmaci inserendo informazioni relative agli organi bersaglio ed alle indicazioni terapeutiche. Sul fronte dell'apprendimento automatico Sul fronte dell'apprendimento automatico, il dataset arricchito potrebbe essere testato con modelli di Deep Learning o Graph Neural Networks (GNN) per sfruttare direttamente la struttura a grafo dell'ontologia ChEBI anziché affidarsi alla sola vettorizzazione dei testi semantici.

Infine, il motore di inferenza Prolog custom potrebbe essere esteso per supportare la programmazione logica probabilistica (es. ProbLog) o la logica fuzzy. Questo consentirebbe al ragionatore simbolico di gestire non solo deduzioni binarie (presenza o assenza di un ruolo), ma anche la natura intrinsecamente incerta, parziale e probabilistica dei dati biologici e clinici, migliorando la fedeltà e la ricchezza delle spiegazioni fornite all'utente.